

More Pie for the Little Typer

Accessible, Extensible, and AI-Guided Proof Education

Anonymous Author(s)

Abstract

Pie, the dependently typed teaching language of *The Little Typer*, is pedagogically near-ideal: just enough to teach dependent types and proof, and no black boxes. Yet its original implementation imposes barriers of its own: a heavyweight local setup, no interactive feedback, a fixed set of built-in types, and proofs written as flat terms with no incremental workflow. We reimplement Pie from scratch in TypeScript and add three extensions that make it more accessible and extensible for learners. *Tactics* build proofs by goal-directed steps yet extract an ordinary, reusable term; *named holes* let incomplete programs type-check and report each goal's type and context; and user-defined *inductive types* come with automatically generated eliminators. An interactive proof canvas renders these tactic proofs and lets a learner build them by direct manipulation. On this core we add THE-TOOL¹, an AI assistant: a self-supervised fine-tuned tactic predictor and a Socratic hint service that explains rather than solves. We report the design, worked examples, and a preliminary evaluation of how the system overcomes the limitations of Pie's original implementation. The result is a browser-based, extensible, AI-guided proof environment built on a minimal, transparent core.

Keywords: theorem proving, dependent types, Pie, computing education, educational programming languages, proof visualization, large language models, cognitive load, scaffolding

1 Introduction

Testing reveals bugs but never certifies their absence; the rigorous alternative is theorem proving—a machine-checked proof that a program meets its specification. Yet theorem proving has a steep learning curve, and good educational tooling for it is scarce.

The provers that dominate practice—Coq, Lean, Isabelle—are industrial-strength but pedagogically forbidding. Using them well demands two kinds of expertise at once: mathematical maturity to structure a proof and functional-programming skill to drive the system. For a newcomer these barriers reinforce each other, and much of the difficulty is incidental to the mathematics—it lives in the tactic language, the library, and the elaborator.

¹tool name anonymized

A natural response of a newcomer is to point a large language model at an open proof. A proof assistant would reject any AI-generated invalid proofs, so the danger is not that invalid proofs slip through. The problem is that AI does nothing for the real barrier—the steep learning curve and the lack of educational scaffolding—and might even deepen it. A stuck, unscaffolded learner simply asks the model for the answer and offloads the very reasoning they came to acquire. The pull is stronger in a large system: against a library like Lean's Mathlib (over 100,000 lemmas) a general model reaches for automation the learner cannot inspect and invents plausible-looking but non-existent lemmas and tactics [18, 23], so the learner cannot even tell a good suggestion from a bad one. The missing piece is educational scaffolding, not a more powerful solver.

What education needs is therefore different from what experts need. The first requirement is minimal sufficiency: a prover should contain just enough to teach dependent type theory and machine-checked proof and nothing more. Then its whole semantics fits in a learner's head, and no incidental feature competes for attention. This is exactly where industrial provers fail: the barrier is one of excess. Two further requirements follow: the prover should be transparent, with no black boxes, so the few features it does have are fully inspectable; and it should be able to grow with a curriculum without abandoning either property. These criteria are not merely aesthetic; they follow directly from how people learn (§5.1).

Pie, the dependently typed language of *The Little Typer* [9], is the rare language that embodies minimal sufficiency by design: it has only the constructs needed to teach the subject and keeps every one of them transparent. But embodying the criteria on paper is not enough. Pie's gaps are not in its calculus but in everything around it. It runs only through a local Racket setup [5], so a learner cannot simply open it nor a teacher deploy it. It has no interactive, goal-directed workflow, so there is no way to make partial progress or see what to do next. It ships only built-in types, so a learner cannot introduce the very structures a course is about. And it offers no guidance when a learner is stuck. The work reported in this paper aims to close these gaps.

Our approach. We address these shortcomings with a system structured, end to end, around three goals—**accessibility**, **extensibility**, and **AI guidance**—layered over a clear hierarchy:

Pie → Language extensions → AI-guided wrapper.

Accessibility and extensibility each have two dimensions. At the system level they concern the implementation and its ecosystem—can the system be installed, run, distributed, embedded, extended, and maintained? At the language level they concern what a learner or teacher can do within the language—can a newcomer make progress, and can the language grow to fit a course? AI guidance is the top layer: a wrapper that depends on the accessible, extensible foundation beneath it and uses it to support learning.

A single tenet keeps all of this faithful to minimal sufficiency: we extend the workflow and the reach of the language without enlarging the minimal core a learner must understand. In our work, we follow the language extensions proposed in Appendix A of *The Little Typer* [9] (“The Way Forward”) by adding inductive datatypes and proof tactics to the language. As we show, tactics and named holes add an interaction layer that still produces ordinary Pie terms (§3.2); inductive types replace a fixed set of built-in types with one general mechanism, a uniformity gain rather than a complication (§4.2); and the AI layer lives entirely outside the language (§5). The core a newcomer must learn stays as small as in the book.

Contributions. This paper presents a design and a system, with preliminary evidence for our design goals; we do not yet show learning gains or full metatheoretic soundness. Concretely, one contribution per theme, plus an early evaluation:

- **Accessibility** (§3): a ground-up reimplementa-tion of Pie in TypeScript that runs in the browser with zero installation (system-level), with goal-directed tactics, named holes, and an interactive proof canvas that are *designed* to make proof construction incremental, navigable, and visible (language-level).
- **Extensibility** (§4): a modular, web-native architecture that is straightforward to extend and embed (system-level), and a data construct for user-defined inductive types with generated eliminators (language-level, §4.2).
- **AI Guidance** (§5): THE-TOOL, a wrapper over this core that adds a self-supervised fine-tuned tactic predictor and a Socratic hint service—each shaped by a learning-science principle and constrained, by design, to explain rather than to solve.
- An *early* evaluation—a held-out tactic-predictor study and an informal user study ($n \approx 20$)—and a candid discussion of what it does and does not show (§6, §7).

2 Background: Pie and *The Little Typer*

Pie is a small, education-oriented dependently typed language: types are ordinary values, so they may be computed and may mention terms. By the Curry–Howard correspondence, type inhabitation is the same activity as proof. A learner

```
(claim + (-> Nat Nat Nat))
(define +
  (lambda (n j)
    (iter-Nat n j (lambda (sum) (add1 sum))))))

;; For every n, n + 0 = n.
(claim step
  (Pi ((n Nat)) (= Nat (+ n 0) n)))
(define step
  (lambda (n)
    (ind-Nat n
      (lambda (k) (= Nat (+ k 0) k)) ;; motive
      (same 0) ;; base
      (lambda (n-1 ih) ;; step:
        (cong ih (+ 1)))))) ;; lift IH across add1
```

Figure 1. A complete proof in plain Pie: $n + 0 = n$ for every n . `claim` states the theorem; `define` supplies a witness that inhabits its type, written as one whole term. §3.2 revisits this same theorem with a named hole.

works with two top-level forms—`claim` states the type of a name, and `define` supplies a term of that type.

As a first proof, consider that adding zero on the right leaves a number unchanged (Figure 1). The `claim` states the theorem—for every n , $(+ n 0)$ equals n —and its type $(\text{Pi } ((n \text{ Nat})) \dots)$ is a dependent function type: a universally quantified statement whose body may mention the argument. The `define` supplies a witness, a function returning a proof for each n ; by Curry–Howard that inhabitant is the proof. It proceeds by induction with the eliminator `ind-Nat`—a motive, a base case $((\text{same } 0))$, as $(+ 0 0)$ computes to 0 , and a step that lifts the induction hypothesis across `add1` with `cong`. Two things stand out: the proof is an ordinary term, with nothing hidden; and the learner must produce that whole term in one go, with no way to make partial progress or inspect the goal mid-construction. Closing that gap is what the next sections are about.

Throughout, we use *goal* for a type to be inhabited and *proof* for a term that inhabits it. Pie checks terms *bidirectionally* and decides equality up to normalization, so definitional equalities (e.g. that $(+ 1 n)$ computes to $(\text{add1 } n)$) hold automatically. Programs and proofs are built with *eliminators*—`ind-Nat`, `ind-List`, and so on—which are the language’s only means of recursion and case analysis and which make every inductive argument explicit. This handful of constructs is the whole language—there is no standard library, no automation, and no implicit arguments—none of the incidental machinery a learner would otherwise have to master alongside the ideas. Critically for teaching, Pie also has no black boxes: every construct is fully inspectable, so a finished proof is a term the learner can read in full. The next three sections take this minimal calculus as given

and address its surrounding gaps in turn: accessibility (§3), extensibility (§4), and AI guidance (§5).

3 Accessibility

3.1 System-Level: A Browser-Native Implementation

Before this work, a learner had to install Racket [8] and Dr-Racket [7] and configure the pie language to use Pie [5]; this confined Pie to a pre-configured desktop and was a real barrier for an introductory audience. We reimplement Pie from scratch in TypeScript.² Because TypeScript compiles to JavaScript, the entire prover—reader, type checker, and evaluator—runs client-side in any browser with zero installation, on desktops and on the tablets now common in classrooms: a learner is one URL away from their first proof.³ The same property makes the system trivial to distribute (it is a static web page) and to embed in other web-based learning environments without per-machine setup.

3.2 Language-Level: Tactics and Named Holes

A running prover is not yet a learnable one. As presented in the book, Pie requires a learner to write a complete proof term in one go, with no notion of partial progress—and that is exactly where novices stall. Two language-level additions make proving incremental and navigable. Both are interaction mechanisms we provide on top of the calculus; neither changes the trusted core, and neither (unlike the inductive types of §4.2) asks the learner to extend the language.

Goal-directed tactics. In Pie a proof is its term: the learner must write the complete inhabitant of a type at once, understanding the goal but facing a blank page with no way to decompose it. We add a goal-directed tactics layer, introduced by a new `define-tactically` declaration, that lets a proof be built one step at a time (Figure 2). This implements the `define-tactically` sketch of *The Little Typer*’s Appendix A [9]. A tactic script runs against a *proof state*: a tree of *goals*, each carrying a local context and a type to inhabit. Each tactic transforms the state—refining the current goal, possibly spawning subgoals—and the system tracks open branches so that a then combinator can dispatch a tactic to each subgoal. The current vocabulary covers *introduction* tactics (`intro` for function types, `exists` for Σ , and `go-Left/go-Right` for sums) and the *exact-term* tactic `exact`; an *eliminator* for each of Pie’s types (`elim-Nat`, `elim-List`, `elim-Vec`, `elim-Either`, `elim-Equal`, `elim-Absurd`, and `split-Pair` for pairs); *equational* reasoning (`symm`, `trans`, `cong`, `rewrite`); and *structural* combinators (`then`, `apply`).

What distinguishes our tactics from a proof script is *proof-term extraction*. Each goal records how to build its term from

²The existing implementations—the book’s Racket package and a Haskell implementation [4]—both require a local toolchain.

³The full browser-based implementation is open source; an anonymized copy for review (source and a runnable build) is available at <https://anonymous.4open.science/r/anonymized-27534876>.

```
(claim identity
  (Pi ((A U) (x A)) A))

(define-tactically identity
  ((intro A)
   (intro x)
   (exact x)))

;; the extracted term is an ordinary value, usable
downstream:
(identity Nat 5)    ;; ==> 5 : Nat
```

Figure 2. A tactic script and its use. `define-tactically` builds the proof by goal-directed steps, but extracts a normal Pie term, so `identity` can be applied like any other definition.

the terms of its children; when the proof state is complete, the tree is folded bottom-up into an ordinary core Pie term. A tactically defined name is therefore indistinguishable from a hand-written one and can be applied in later computations, as in Figure 2 where `identity` is used as a normal function. Tactics thus add a gentler interaction mode for language-level accessibility, without changing the trusted core. The artifact a learner produces is still a plain Pie term they can read. The goal tree a tactic script builds is also what the visual canvas of the next subsection renders (§3.3).

Named holes. The second barrier to progress is that a proof term is all-or-nothing: the step proof of Figure 1 does not type-check until its whole body is in place. Pie’s single hole keyword, `TODO`, lets a learner leave a placeholder that type-checks and reports its expected type and local context. But every hole shares the one name, so in a proof with several open goals neither the learner nor any tool can tell the reports apart. We extend the parser so each hole carries a distinct *name* (`TODO-<name>`). A program can then carry several open goals, each reporting its own expected type and context (Figure 3). Named holes make proving incremental and give every open goal a stable identifier that the AI layer of §5 can address. We propose named holes to add an incremental interaction layer, following established concepts in interactive theorem provers such as Agda [16] and Idris [3]; the contribution is not the concept of named holes but their inclusion in Pie’s teaching-oriented, browser-native environment.

3.3 Visual Proof Construction

A proof’s structure is a tree, but Pie’s linear text hides it. A learner cannot see which subgoals a step produced or which remain. By cognitive load theory [19], that is extraneous load—effort spent decoding notation rather than learning the proof. We remove it with an interactive proof canvas (Figure 4).

```

331 (claim step
332   (Pi ((n Nat)) (= Nat (+ n 0) n)))
333 (define step
334   (lambda (n) TODO-goal)) ;; a NAMED hole
335 ;; type-checks; reports at hole TODO-goal:
336 ;;   Expected: (= Nat (+ n 0) n)
337 ;;   Context:  n : Nat
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385

```

Figure 3. A named hole. Pie’s holes all share the single name TODO; our parser assigns each hole a distinct name (TODO-⟨name⟩), so a program with several open obligations reports each one separately with its own expected type and local context.

The canvas renders the tactic proof state as a graph. Each node is a goal, showing its type and local context; each edge is the tactic that produced the goal below. On request, an LLM API paraphrases a goal’s formal type into natural language—the goal text in Figure 4—and “Show Pie syntax” toggles back to the Pie term. A learner does not just read this graph but builds the proof in it: dragging a tactic onto a goal applies it, and the new subgoals appear as nodes. The kernel runs in a background worker, so the editor stays responsive while it checks each step.

The canvas and the text stay in sync. At any point the system exports the graph as an ordinary define-tactically script (§3.2), so a learner can read the proof as a tree or as a term. Visualization thus makes the structure visible without hiding the term; it complements Pie’s transparency. To keep large proofs manageable, the canvas also supports panning, a minimap, and collapsing finished branches.

The canvas currently visualizes proofs built by tactics. Visualizing named holes as addressable goals, and depicting the structure of user-defined inductive types, are future work (§9).

4 Extensibility

Extensibility likewise has two sides, but they concern different people. *System-level* extensibility concerns the implementers: can the system be extended, maintained, and embedded as needs change? *Language-level* extensibility concerns the users: can a learner or teacher grow the language itself with the structures a course needs? Our TypeScript foundation delivers the first; user-defined inductive types deliver the second.

4.1 System-Level: A Modular, Web-Native Architecture

The implementation follows the pipeline of *The Little Typer* closely (Figure 5), which keeps it readable alongside the book and easy to audit for faithfulness. Source text is first read by a general-purpose Scheme reader into S-expressions, then a Pie-specific declaration parser turns each top-level form into

a typed *source* AST node. Type checking is *bidirectional*: every source node implements a *synthesize* judgement (infer a type) or a *check* judgement (check against an expected type), elaborating to a small *core* language. Core terms are given meaning by *normalization by evaluation* (NbE): they are evaluated into semantic values and then *read back* into normal forms, which is how the checker decides type and term equality. A top-level driver threads a typing context through the five declaration forms the system supports: *claim* (assert a type), *define* (give a term), a *sameness* check, a *datatype* definition (§4.2), and a *tactic-based* definition (§3.2).

Because the kernel is modular, statically typed, and guarded by 80 Jest test suites, it is straightforward to extend rather than rewrite. The tactics and named holes of §3.2 and the inductive types of §4.2 were each added as localized extensions—a parser case, a source-node class, and the matching checking and evaluation rules. The same properties—one mainstream language across the stack, a mature package ecosystem, and browser embedding—let others maintain and embed the prover, and lower the friction of plugging in *external* tools, from editor services to the AI components of §5. The very same kernel already backs a VS Code language server, an online playground, and THE-TOOL: system-level extensibility is what makes the AI-guided layer possible at all. The test corpus is also a first-class artifact—the source of our tactic-prediction training data (§5.2).

4.2 Language-Level: User-Defined Inductive Types

System-level extensibility serves implementers; language-level extensibility must serve learners. As the book leaves it, Pie offers only a fixed repertoire of types—*Nat*, equality, pairs, *List*, *Vec*, *Either*, and a few more—each baked in with its own bespoke eliminator. A learner cannot introduce the structure a course is actually about: a less-than relation, a typed syntax tree, an evenness predicate. We add a single data declaration for user-defined *inductive families* (Figure 6). A declaration names zero or more *parameters* and *indices*, lists its constructors, and names an eliminator. The system elaborates it, adds the type and its constructors to the context, and *generates the eliminator automatically*, so *ind-MyVec* becomes available with no further work.

This addition *simplifies* Pie. One general mechanism replaces the ad-hoc set of built-in types-with-eliminators. That mechanism reaches nearly every discrete structure an introductory course uses—relations, orderings, trees, parity—so the language grows with the curriculum while the learner still has just one construct to understand. The addition is opt-in: a course introduces only the families it needs. It keeps Pie’s no-black-boxes spirit—the generated eliminator is an ordinary dependent recursor, and proofs over user types look like proofs over built-in ones. It also follows the book’s own trajectory: Appendix A of *The Little Typer* [9] proposes exactly this extension. This is the extensibility the *user* wields, without enlarging the core.

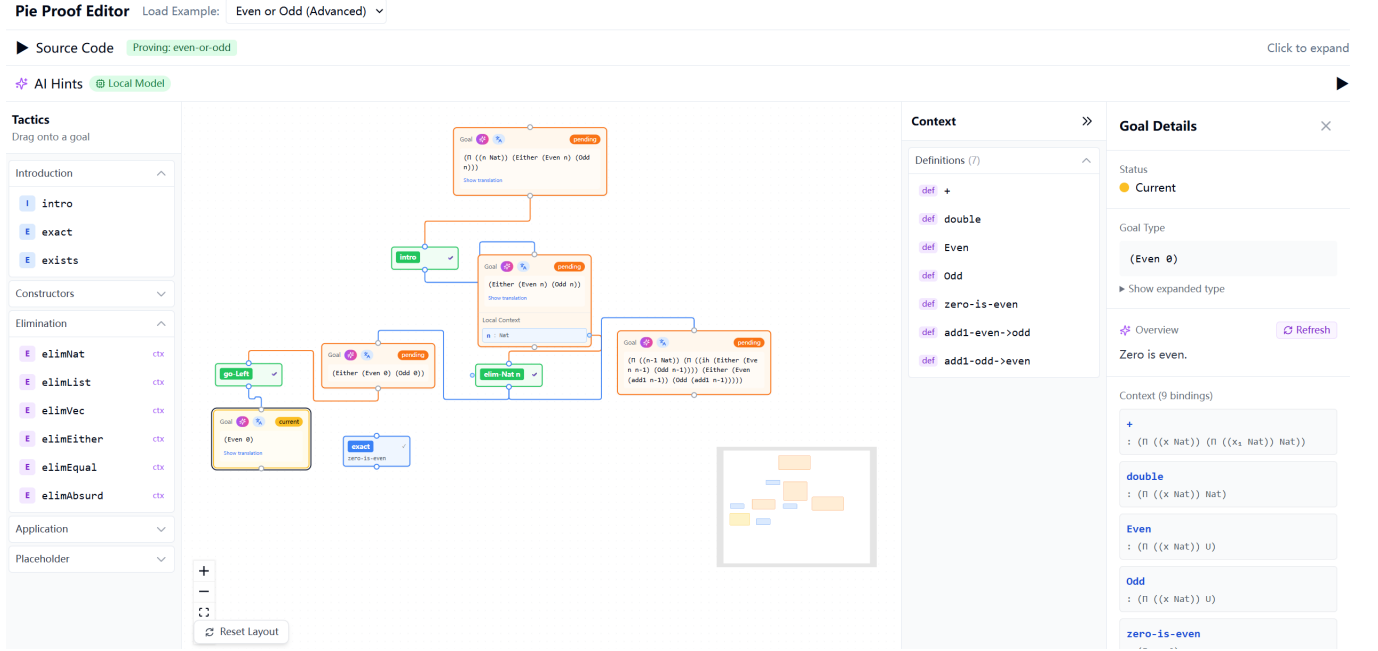


Figure 4. The interactive proof canvas, proving that every natural number is even or odd. Each goal node shows its type and local context; each edge is an applied tactic; a learner drags tactics from the palette (left) onto a goal, and the canvas exports an equivalent define-tactically script. A minimap and a goal-details panel aid navigation, and a badge marks each goal’s status (*pending*, *current*, or *done*).

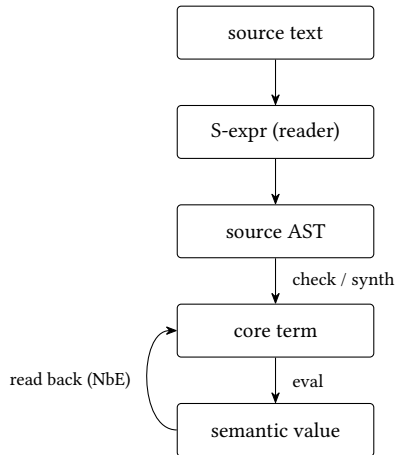


Figure 5. The kernel pipeline, mirroring *The Little Typer*: source text is read to S-expressions, parsed to a typed source AST, elaborated by a bidirectional checker to a core term, and given meaning by normalization by evaluation (evaluate to a value, read back to a normal form). Its modularity is what makes the system extensible (§4.1).

Elaboration and eliminator generation. A data declaration separates *parameters*, fixed across the whole family, from *indices*, which may vary between constructors. In Figure 6, `MyVec` has one parameter (the element type `E`) and one

index (the length `n`). Each constructor is checked in a context extended with the parameters, and its return type is checked against the family being defined. To generate the eliminator, elaboration partitions each constructor’s arguments into *non-recursive* and *recursive* ones (occurrences of the type being defined). The recursive arguments induce the induction hypotheses that the eliminator’s method receives—the `ih` parameter in Figure 6. The generated eliminator takes a motive over the indices and target, one method per constructor, and computes by the expected dependent-recursion scheme.

Soundness. The data construct is the one addition that extends the language itself rather than the workflow; we call the result *Pie’* (Pie with user-defined inductive types) and keep *Pie* for the book’s minimal core. A datatype mechanism is sound only if it is *strictly positive*: the type being defined must not occur in a constructor argument to the left of an arrow (a negative occurrence), which would otherwise let one derive `Absurd` and collapse the logic. A full metatheoretic treatment of *Pie’*—the soundness of data declarations and their generated eliminators—is left to future work.

5 AI Guidance: THE-TOOL

The accessible, extensible core of §3–§4 is a good place to learn, but a learner working alone still gets stuck. THE-TOOL [2] is an AI-guided wrapper that sits *above* the language and its extensions and depends on both. It depends

on accessibility: it delivers guidance in the same zero-install browser runtime, in place, next to the proof. And it depends on extensibility: it reasons over the structured proof state that the extensions expose—the goals and subgoals of a tactic script, the expected type and context of a named hole, the constructors and eliminator of a user-defined type. We make AI guidance a first-class layer. Every component answers a learning-science principle and is designed to guide the learner without producing the proof for them.

5.1 Design Rationale: Theory Drives the Design

Like the proof canvas of §3.3, which answers cognitive load theory [19], each AI component of THE-TOOL answers a learning-science principle. *Scaffolding within the zone of proximal development* [20, 21]—support a learner to do what they cannot yet do alone, without doing it for them—motivates the Socratic hint service (§5.3). *Cognitive apprenticeship* [6]—make expert reasoning visible and coach the next step—motivates the tactic predictor (§5.2) and the Socratic prompting style. The *expertise-reversal effect* [14]—heavy support helps novices but hurts once schemas form—shapes how the hints must fade (§5.3).

```
;; Length-indexed vectors (Pie has Vec built in).
;; Parameter E (element type) is fixed;
;; index n (the length) varies.
(data MyVec ((E U)) ((n Nat))
  (myVecNil () (MyVec (E) (zero)))
  (myVecCons
    ((k Nat) (head E) (tail (MyVec (E) (k))))
    (MyVec (E) ((add1 k)))))
ind-MyVec

(claim one-vec (MyVec (Nat) ((add1 zero))))
(define one-vec
  (myVecCons zero (add1 zero) (myVecNil)))

;; a function via the generated eliminator:
(claim vec-to-nat
  (Pi ((n Nat)) (-> (MyVec (Nat) (n)) Nat)))
(define vec-to-nat
  (lambda (n v)
    (ind-MyVec v
      (lambda (len vec) Nat)
      zero
      (lambda (k h t ih) (add1 ih)))))
```

Figure 6. A user-defined inductive family from our test suite: length-indexed vectors, a type `Pie` provides built in. `MyVec` has a *parameter* `E` (fixed across the family) and an *index* `n` (the length, which varies); the system generates `ind-MyVec`, used here in `vec-to-nat`. `data` lets learners extend the language with the structures a course needs.

```
{ "theoremName": "n+0=n",
  "theoremType":
    "(Pi (n Nat) (= Nat (iter-Nat n (the Nat 0)
      (lambda (x) (add1 x))) n))",
  "stepIndex": 0,
  "globalContext": [{"name": "+", "type": "(Pi (x Nat) (Pi
    (y Nat) Nat))"}],
  "localContext": [],
  "goal": "(Pi (n Nat) (= Nat ...))",
  "tactic": "intro n" }
```

Figure 7. One auto-generated training example (abbreviated): a goal with its global and local context, paired with the tactic that advanced it. Every successful proof step yields one such line, with no manual labeling.

5.2 A Self-Supervised, Fine-Tuned Tactic Predictor

Because every proof in our test corpus runs through the tactic engine, the interpreter records each successful step as a training example, with no manual labeling. Each example is one JSONL line: the goal with its global and local context, paired with the tactic that advanced it (Figure 7). The artifact that teaches humans thus doubles as a dataset for teaching a model. Each proof contributes one example per tactic step, giving 3,794 steps from 948 proofs across 12 tactic types. We fine-tune Qwen2.5-Coder-7B-Instruct [13] on this data with a 4-bit low-rank adapter [12] to predict the next tactic from a goal-plus-context, and evaluate it on *novel* theorems held out of training (§6.1).

5.3 Socratic LLM Hint Service

Novices most often stall not on executing a step but on choosing one. When a learner asks for help on an open goal, THE-TOOL assembles the goal’s type, its local assumptions, and the available definitions. It then issues a hint request grounded in exactly the state the learner is inspecting. Two back-ends serve such requests: the fine-tuned predictor of §5.2, which proposes a next tactic, and a general-purpose LLM accessed through its API, which produces a natural-language explanation.

The predictor can still propose an ill-typed or non-existent tactic—the hallucination risk of §1. THE-TOOL runs each suggested tactic through the kernel before showing it and surfaces only those the proof engine accepts, so a learner never sees an invalid hint.

The pedagogically essential decision is what the hint is *allowed* to say. A general assistant asked for a `Pie` proof would simply emit the answer—the failure mode of §1, in which the learner offloads the reasoning they came to acquire. Our prompt instead constrains the model to *teach*: explain why the goal has the shape it does and which form of reasoning applies (“the goal is a Π -type, so introduce the bound

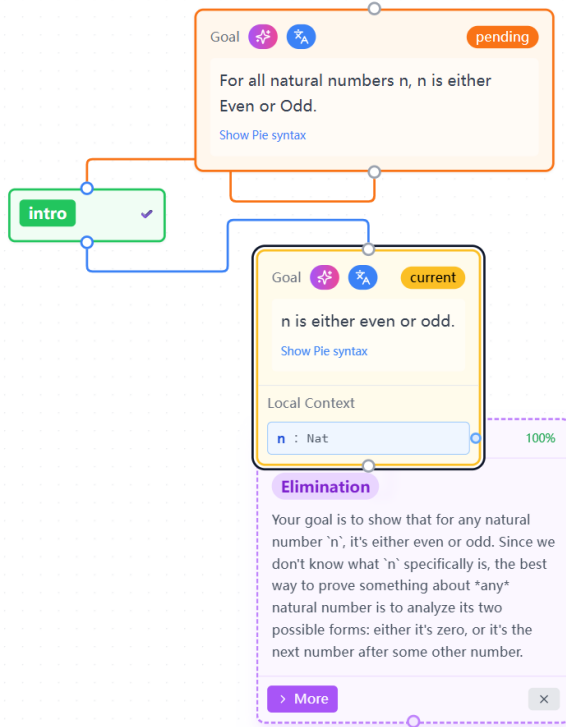


Figure 8. An AI-suggested tactic appears as a distinct, dashed “ghost” block below the current goal, set apart from real tactic nodes. It is disclosed progressively: shown here at its first (category) level, with a plain-language rationale and a “More” button for the next level. The learner accepts a suggestion—turning it into a real tactic node—or dismisses it; the proof is unchanged until then.

variable first”), without supplying the term. This makes the assistance Socratic by construction. It also makes it *fade*, as the expertise-reversal effect [14] demands: hints are pulled per goal rather than pushed, they explain rather than solve, and the learner always performs the step. Strong support sits in the zone of proximal development early; autonomy is the default as the learner improves.

A hint never edits the proof. It appears beside the goal as a distinct, semi-transparent *ghost* block (Figure 8), set apart from the real tactic nodes. The hint is disclosed progressively—first a category of reasoning, then a specific tactic, then its full parameters—so a learner can stop at the level of help they need. Accepting the ghost turns it into a real tactic node; dismissing it leaves the proof untouched. The block makes “pulled, not pushed” and “the learner performs the step” concrete.

6 Evaluation

Our evaluation touches two of the three themes directly. *Accessibility*: does the system lower the barrier for learners? We address this with a user study. *AI guidance*: is the predictor

reliable enough to drive hints over a curriculum? We address this with a predictor study. *Extensibility* we argue qualitatively, through the worked examples of §4. Both studies are preliminary, and we are explicit about what they do and do not show.

6.1 Tactic Predictor (AI Guidance)

We evaluate the predictor on a *held-out* set of 157 novel theorems—706 tactic steps across twelve proof patterns, none seen in training. Because the test theorems are new, this measures *generalization* rather than memorization.

We first score each step by *string match* against the reference proof (Table 1): *exact match* (tactic and arguments identical), *tactic head* (the right *kind* of tactic—e.g. `elim-Either`—regardless of arguments), and *category* (the right family of reasoning). The model chooses the right kind of tactic 92.8% of the time and the right category 95.6%; full exact match is 72.9%.

String match understates real ability, because a hint engine needs a *usable* move, not the recorded one. So we also validate predictions against the kernel: a step is *kernel-valid* if the predicted tactic parses and the proof engine accepts it (closing the goal or yielding well-typed subgoals). All 706 gold tactics pass this check, and 91.6% of the model’s predictions do—about 19 points above exact match. The gains concentrate exactly where string match was pessimistic: `go-Right` and `apply` predictions are almost always type-valid even when they differ from the reference. Per-step validity is necessary but not sufficient, however—a well-typed tactic can still steer toward an unprovable goal—so we do not read 91.6% as a success rate.

The sound end-to-end measure is *free-running completion*. Starting from the bare theorem, the model proposes a tactic, the kernel applies it, the model sees the new state, and this repeats until the proof closes or the kernel rejects a step. Driven this way the model proves 62.4% of the 157 novel theorems outright—kernel-verified, from scratch—against 40.1% if we demand every step match the reference. A wrong branch (the `go-Right` bias, boolean case splits) simply fails to close, so this figure does not over-credit valid-but-doomed moves. For a hint engine that proposes the next move while the learner drives, 92.8% head accuracy and 91.6% per-step validity are the relevant numbers; 62.4% is a floor on what the model can do unaided. How far any of this extends to theorem families well outside the training distribution is untested (§7).

The predictor is a 4-bit QLoRA adapter (rank 32, α 64) over Qwen2.5-Coder-7B-Instruct, trained in about 90 minutes on a single laptop GPU; full data, training, and evaluation scripts are in Appendix A.

6.2 User Study (Accessibility)

We gathered informal feedback from 19 participants at an institutional project showcase, who tried the proof canvas

Table 1. Held-out evaluation of the fine-tuned Qwen2.5-Coder-7B-Instruct predictor on 157 *novel* theorems (706 tactic steps), none seen in training. The first three rows are string match against the reference; the last two use the Pie kernel. Gold tactics are 100% kernel-valid.

Metric	Score
Tactic head (right kind of tactic)	92.8%
Category (right family of reasoning)	95.6%
Exact match (tactic + arguments)	72.9%
Kernel-valid step (type-checks & applies)	91.6%
Free-running completion (proof closed)	62.4%

on small examples and commented freely; we collected no quantitative measures, so we read it as direction, not a result.

The reactions were mixed. One participant, already familiar with theorem provers, found THE-TOOL noticeably more approachable than the provers they knew, crediting the visible proof state and the grounded hints—though, from an experienced user, this speaks to ergonomics more than to whether a newcomer gets further. Another, with less background, met the opposite: the interface felt primitive, it was unclear how to apply the tactic blocks, the attempt hit bugs, and even the even-or-odd theorem felt too hard a first target. The feedback points to onboarding—getting-started guidance, clearer blocks, a gentler first theorem, and polish—which we treat as an actionable agenda, not evidence of learning gains.

7 Discussion and Limitations

We organize the main limitations by the same three themes, since each marks where that theme’s contribution is still incomplete.

Accessibility. Our user study is small, single-session, and showcase-based; it indicates first-impression approachability, not learning outcomes, and participants found the interface still primitive. A real classroom evaluation is needed (§9).

Extensibility. Pie’, our data-extended language (§4.2), has a lightweight metatheory: we have not mechanized a soundness proof for its data declarations or their generated eliminators, including the strict-positivity argument they rely on. The named-hole extension, likewise, is not yet surfaced graphically (§9).

AI guidance. The predictor generalizes to novel theorems and proves 62.4% of them end-to-end (§6.1), but that leaves most proofs unfinished unaided, and it has known weak spots—a go-Right bias and boolean case splits—where a type-valid but wrong choice derails a branch. Its behavior on theorem families far from the training distribution is untested. An LLM hint can also mislead even when fluent; our mitigations are structural (explain not solve, pulled per

goal, learner performs every step), but we have not yet measured whether they prevent over-reliance, which requires a longitudinal study.

8 Related Work

Educational PL tooling. SPLASH-E and adjacent venues have a rich line of tools and languages built for learners: visualizations of automata and Turing machines [15] and the design of educational languages such as Hedy [11]. We share this stance but target a topic—machine-checked dependently typed proof—that has received far less educational tooling, and we make accessibility, extensibility, and AI guidance the explicit axes of the design.

Proof assistants and their pedagogy. Coq, Lean, and Isabelle are the standard for serious proof and have been used to teach it. Our argument (§1) is that their barrier for beginners is one of *excess*, not deficiency: vast libraries, powerful opaque automation, and heavy elaboration. A minimally sufficient, transparent language is better positioned to teach the underlying ideas, adding the features a course needs deliberately rather than all at once.

Machine learning for tactic prediction. Learning to predict proof steps is well studied—e.g. CoqGym [22], TacticToe [10], and neural proof generation [17]. These target expert-scale libraries and general proving. Our setting differs in two ways: the training data is *self-supervised* from the same curriculum that teaches humans, requiring no manual labeling; and the goal is reliability over a *bounded* educational corpus rather than open-ended generalization.

9 Future Work

The three themes also chart the road ahead. Beyond the metatheoretic hardening of Pie’ and a broader predictor evaluation on theorem families outside the training distribution—both noted in §7—three directions stand out.

Deepening extensibility and AI guidance together. Extending the proof canvas (§3.3) to display *named holes* as addressable goals, and to depict user-defined *inductive types* and their generated eliminators, would let the AI layer reach the full extensible core. The predictor and Socratic hint service could likewise be made hole-aware—suggesting how to fill a specific named hole—and could help a learner choose a constructor or apply an eliminator over a type they have just defined.

Broadening accessibility to new audiences. Our curriculum so far targets the theorems of an introductory type-theory or discrete-mathematics setting. Because Pie keeps its full logical machinery visible and its surface small, the same approach could reach *earlier* audiences—secondary- and high-school mathematics, where the goal is less to prove

software correct than to make the idea of a rigorous, checkable proof tangible. Adapting the curriculum, the inductive-type library, and the hint style to school-level proof is a natural next step.

Classroom evaluation. Our present study is small, single-session, and showcase-based. We plan a real classroom deployment in an introductory university course next term. It would support a longitudinal, controlled evaluation of *learning outcomes*—not just first-impression approachability—including completion rates with and without THE-TOOL and an analysis of common errors before and after use.

10 Conclusion

An elegant teaching language is not enough if a learner cannot run it, cannot grow it, and cannot get unstuck in it. We took Pie—valued for its minimal sufficiency, just enough to teach the subject and nothing more—and rebuilt everything *around* that core without enlarging it. For **accessibility**, a faithful TypeScript reimplementation puts the whole prover one URL away, and goal-directed tactics, named holes, and a visual proof canvas make proving incremental and visible while still producing ordinary Pie terms. For **extensibility**, a modular web-native architecture is easy to extend and embed, and user-defined inductive types replace the fixed set of built-in types with one general mechanism that reaches nearly every discrete structure a course needs. For **AI guidance**, THE-TOOL layers a self-supervised tactic predictor and a Socratic hint service over that foundation, constrained by design to explain rather than to solve. Throughout, we extended the workflow and reach of the language while keeping the core a newcomer must learn as small as in the book. Our evidence is early but encouraging—informal feedback on approachability, and a held-out predictor that proves 62.4% of novel theorems end-to-end through the kernel—and we are explicit about what remains, above all a controlled classroom study of learning outcomes. We offer the design and the artifact as a foundation for that work. The implementation, model, and reproduction scripts are available for review (Appendix A).

References

- [1] Anonymous. 2026. Software repository (anonymized for review). <https://anonymous.4open.science/r/anonymized-27534876>.
- [2] Anonymous. 2026. THE-TOOL—high-level description. In *a computing conference 2026, extended abstract*.
- [3] Edwin Brady. 2013. Idris, a General-Purpose Dependently Typed Programming Language: Design and Implementation. *Journal of Functional Programming* 23, 5 (2013), 552–593.
- [4] David Thrane Christiansen. 2018. Software repository of pie-hs. <https://github.com/david-christiansen/pie-hs>.
- [5] David Thrane Christiansen and Daniel P. Friedman. 2021. The Pie Reference. <https://docs.racket-lang.org/pie/>.
- [6] Allan Collins, John Seely Brown, and Susan E. Newman. 1989. Cognitive Apprenticeship: Teaching the Crafts of Reading, Writing, and Mathematics. In *Knowing, Learning, and Instruction: Essays in Honor of*

- Robert Glaser*, Lauren B. Resnick (Ed.). Lawrence Erlbaum Associates, 453–494.
- [7] Robert Bruce Findler, John Clements, Cormac Flanagan, Matthew Flatt, Shriram Krishnamurthi, Paul Steckler, and Matthias Felleisen. 2002. DrScheme: A Programming Environment for Scheme. *Journal of Functional Programming* 12, 2 (2002), 159–182.
- [8] Matthew Flatt and PLT. 2010. *Reference: Racket*. Technical Report PLT-TR-2010-1. PLT Design Inc. <https://racket-lang.org/tr1/>.
- [9] Daniel P. Friedman and David Thrane Christiansen. 2018. *The Little Typer*. MIT Press.
- [10] Thibault Gauthier, Cezary Kaliszyk, Josef Urban, Ramana Kumar, and Michael Norrish. 2021. TacticToe: Learning to Prove with Tactics. *Journal of Automated Reasoning* 65 (2021).
- [11] Felienne Hermans. 2020. Hedy: A Gradual Language for Programming Education. In *ACM Conference on International Computing Education Research (ICER)*.
- [12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [13] Binyuan Hui, Jian Yang, Zeyu Cui, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).
- [14] Slava Kalyuga, Paul Ayres, Paul Chandler, and John Sweller. 2003. The Expertise Reversal Effect. *Educational Psychologist* 38, 1 (2003), 23–31.
- [15] Marco T. Morazán, Tijana Minic, and Oliwia Kempinski. 2024. Visualizing Composed Turing Machines. In *ACM SIGPLAN International Symposium on SPLASH-E*.
- [16] Ulf Norell. 2007. *Towards a Practical Programming Language Based on Dependent Type Theory*. Ph. D. Dissertation. Chalmers University of Technology.
- [17] Alex Sánchez-Stern, Yousef Alhessi, Lawrence Saul, and Sorin Lerner. 2020. Generating Correctness Proofs with Neural Networks. In *ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*.
- [18] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. 2024. Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean. *arXiv preprint arXiv:2404.12534* (2024).
- [19] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2 (1988), 257–285.
- [20] Lev S. Vygotsky. 1978. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.
- [21] David Wood, Jerome S. Bruner, and Gail Ross. 1976. The Role of Tutoring in Problem Solving. *Journal of Child Psychology and Psychiatry* 17, 2 (1976), 89–100.
- [22] Kaiyu Yang and Jia Deng. 2019. Learning to Prove Theorems via Interacting with Proof Assistants. In *International Conference on Machine Learning (ICML)*.
- [23] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

A Reproduction and Setup

This appendix is supplementary: it builds the system, serves the predictor, runs the frontend, connects them, and reproduces the held-out evaluation of §6.1.

Artifact availability. An anonymized artifact accompanies the submission—the source (interpreter, web frontend, language server) [1], the fine-tuned LoRA adapter (≈320 MB), the training and evaluation scripts, the training data, and the

held-out test set—at <https://drive.google.com/file/d/1SfVUGqSsLwI0UxCQjGtD0Nra9cjd2vHf/view?usp=sharing>. The source repository is also browsable online (anonymized for review) at <https://anonymous.4open.science/r/anonymized-27534876>. The base model [13] is public and fetched at build time, so the bundle ships only the adapter; an included Dockerfile rebuilds the image. The two distributions differ in one respect: the online playground runs the prover, editor, and visualizer in the browser but does *not* host the fine-tuned predictor, whereas the artifact bundle includes it—the LoRA adapter runs locally against a self-hosted model server, giving the full AI-guided workflow.

Requirements. Node.js 18+ and Yarn for the interpreter and frontend. The predictor needs an NVIDIA GPU (CUDA 12.1+, ≥ 12 GB VRAM), served either from a Docker image built from the included Dockerfile or natively from a Conda environment. A Gemini API key is optional and enables the natural-language features.

Step 1: build the interpreter. Install dependencies, build, and run the test suite:

```
yarn install && yarn build && yarn test
```

Step 2: serve the tactic predictor. The model is Qwen2.5-Coder-7B-Instruct with a 4-bit QLoRA adapter (rank 32, α 64), trained in about 90 minutes on a single laptop GPU; the adapter is ≈ 320 MB (full hyperparameters in the artifact). Serve it natively or from the Docker image; either way it exposes /health and /predict on port 8000.

```
# native (NVIDIA GPU + Conda)
python training/serve.py --adapter <adapter> --port 8000
# or simply: ./training/launch.sh
```

Step 3: run the frontend. Start the Vite dev server:

```
cd web-react && yarn install
yarn dev # localhost:5173
```

The proof editor works with no model—proofs can be built by hand.

Step 4: connect the frontend to the predictor. In the editor’s *AI Hints* panel, set the *Local Model Server* URL to `http://localhost:8000`; the health indicator turns green once the server is reachable. Requesting a hint on a goal now routes through the adapter; each suggested tactic is checked against the kernel before it is shown (§5.3). The hint chain is: adapter prediction, then a Gemini explanation (Step 5), then a rule-based fallback if neither is configured.

Step 5: enable the Gemini features. A Gemini API key (the free tier suffices) powers two features: progressive hint *explanations*—category, then tactic, then full arguments (§5.3)—and goal *translation*, which renders a Pie type in English (§3.3). Paste the key into the *AI Hints* panel (it persists in `localStorage`), or set `VITE_GOOGLE_API_KEY` in `web-react/.env.local`. With both the predictor (Step 2) and the key configured, all of §3.3 and §5 is available.

Training data. `training-data-clean.jsonl` holds 3,794 tactic steps from 948 proofs (normalized goal, context, and tactic per line); a chat-formatted variant is the training input; intro (41%) and exact (32%) dominate the distribution.

Reproducing the held-out evaluation. The held-out set—157 novel theorems, 706 tactic steps (§6.1)—is defined in `eval-holdout.ts`, so no separate data file is needed. With the Step 2 server running, one command reproduces every row of Table 1:

```
npx tsx training/eval-holdout.ts --kernel-validate
```

It reports the string metrics (92.8% tactic-head, 95.6% category, 72.9% exact-match) and the kernel metrics (91.6% of predictions valid, gold tactics 100%; 62.4% closed end-to-end). A lighter Python scorer, `evaluate_offline.py`, computes the string metrics over a smaller even-or-odd subset as a quick offline check.